Flutter InteractiveViewer Widget – Complete Documentation

Overview

`InteractiveViewer` is a Flutter widget that enables **zooming, panning, and scaling** of its child widget.

It is commonly used for:

- Image zoom
- Maps
- Diagrams
- Canvas editors
- Large content viewers

It provides smooth gestures like:

- Pinch to zoom
- Drag to pan
- Scale content

Constructor

```
InteractiveViewer({
  Key? key,
  Widget? child,
  double minScale = 0.8,
  double maxScale = 2.5,
  EdgeInsets boundaryMargin = EdgeInsets.zero,
  bool constrained = true,
  bool panEnabled = true,
  bool scaleEnabled = true,
})
```

Important Properties

child

The widget that will be zoomed and panned.

```
child: Image.asset("assets/image.png")
```

---

minScale

Minimum zoom level.

```
minScale: 0.5
```

---

maxScale

Maximum zoom level.

```
maxScale: 4.0
```

---

boundaryMargin

Allows movement outside normal bounds.

```
boundaryMargin: EdgeInsets.all(20)
```

---

panEnabled

Enable or disable dragging.

```
panEnabled: true
```

scaleEnabled

Enable or disable zoom.

```
scaleEnabled: true
```

Basic Example

```
InteractiveViewer(
  child: Container(
    width: 300,
    height: 300,
    color: Colors.blue,
    child: Center(
      child: Text(
        "Zoom Me",
        style: TextStyle(color: Colors.white),
      ),
    ),
  ),
)
```

Advanced Example

```
InteractiveViewer(
  boundaryMargin: EdgeInsets.all(50),
  minScale: 0.5,
  maxScale: 5,
  child: Image.network(
    "https://picsum.photos/800/800",
  ),
)
```

Real World Example – Image Viewer

```
Center(
  child: InteractiveViewer(
    minScale: 0.1,
    maxScale: 10,
    child: Image.asset("assets/large_image.jpg"),
  ),
)
```

Best Practices

- Use for large content
- Set proper minScale and maxScale
- Use boundaryMargin for better UX
- Wrap large images or canvas

Performance Tips

Avoid using very large widgets without optimization.

Use compressed images.

Full Source Code Example

```
import 'package:flutter/material.dart';

class InteractiveViewerExample extends StatelessWidget {
  const InteractiveViewerExample({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("InteractiveViewer Example"),
      ),
      body: Center(
```

```
        child: InteractiveViewer(
          boundaryMargin: const EdgeInsets.all(20),
          minScale: 0.5,
          maxScale: 4,
          child: Container(
            width: 300,
            height: 300,
            decoration: BoxDecoration(
              color: Colors.blue,
              borderRadius: BorderRadius.circular(20),
            ),
            child: const Center(
              child: Text(
                "Pinch to Zoom",
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 20,
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

When to Use InteractiveViewer

Use when:

- Image zoom needed
- Canvas editor
- Diagram viewer
- Map-like interaction

When NOT to Use

Avoid when:

- Simple static UI
- Small widgets

Summary

InteractiveViewer provides powerful zoom and pan functionality with minimal code.

It is essential for advanced Flutter UI and professional apps.

You can edit and expand this documentation directly.